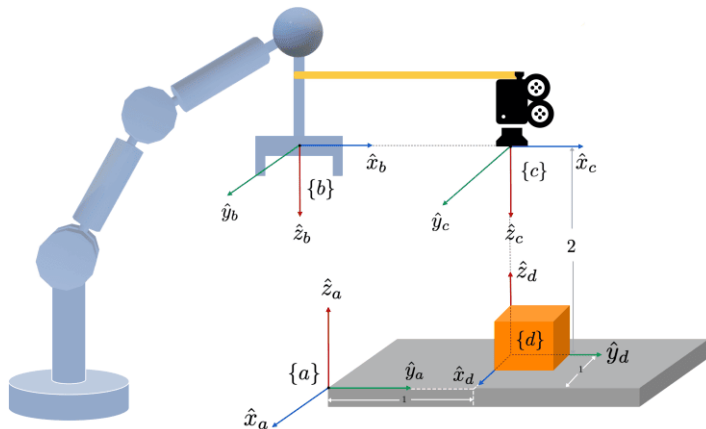


Fundamentals for robot manipulators

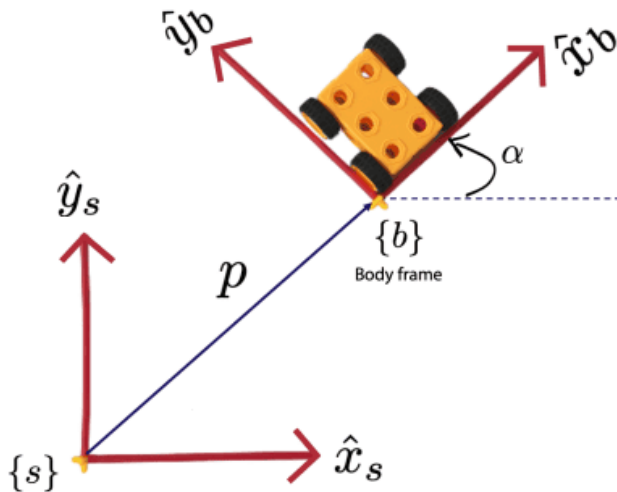
Universidade Federal de Pernambuco
Adrien Durand-Petiteville
`adrien.durandpetiteville@ufpe.br`

- A fundamental requirement in robotics is to represent the position and orientation of objects in an environment

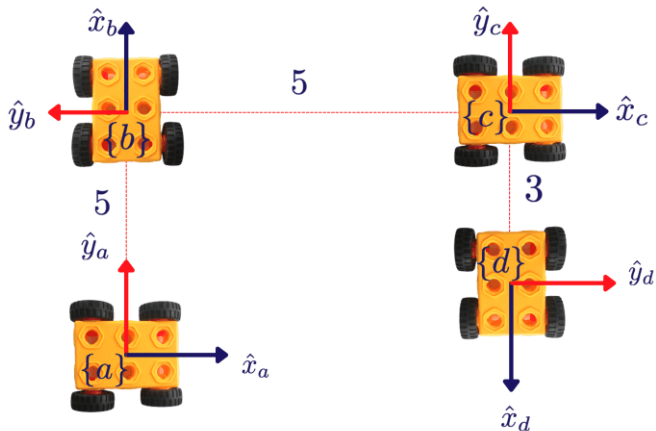


Intro

- A fundamental requirement in robotics is to represent the position and orientation of objects in an environment

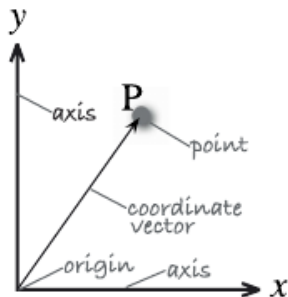


- A fundamental requirement in robotics is to represent the position and orientation of objects in an environment



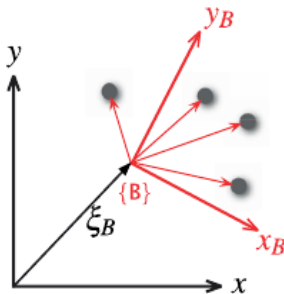
Point

- A point in space can be described by a coordinate vector
- The vector represents the displacement of the point with respect to some reference coordinate frame
- A coordinate frame, or Cartesian coordinate system, is a set of orthogonal axes which intersect at a point known as the origin



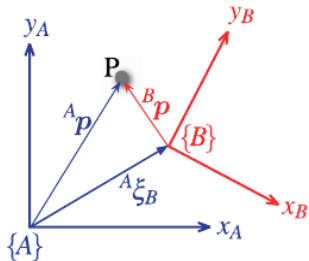
Set of points

- More frequently we need to consider a set of points that comprise some object.
- We assume that the object is rigid and that its constituent points maintain a constant relative position with respect to the object's coordinate frame.
- Instead of describing the individual points we describe the position and orientation of the object by the position and orientation of its coordinate frame $\{B\}$.



Relative pose

- The relative pose of a frame with respect to a reference coordinate frame is denoted by the symbol ξ
- Relative pose ${}^A\xi_B$ describes $\{B\}$ with respect to $\{A\}$
 - The superscript denotes the reference coordinate frame
 - The subscript denotes the frame being described



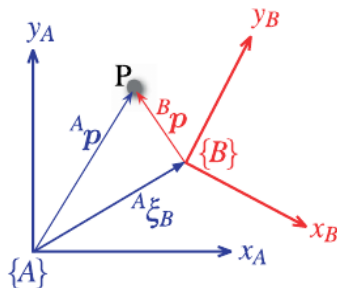
- We could also think of ${}^A\xi_B$ as describing some motion: applying a displacement and a rotation on $\{A\}$ so that it is transformed to $\{B\}$

Point coordinates

- The point $\mathbf{P}$ can be described with respect to either coordinate frame
- Formally we express this as:

$${}^A\mathbf{p} = {}^A\xi_B \cdot {}^B\mathbf{p}$$

- The operator $\cdot$ transforms the vector, resulting in a new vector that describes the same point but with respect to a different coordinate frame.

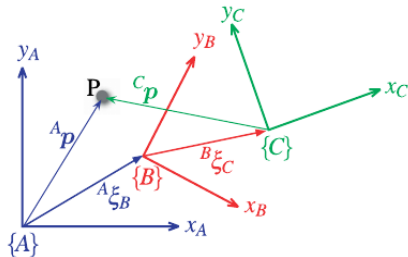


Composition of relative poses

- Relative poses can be composed
- If one frame can be described in terms of another by a relative pose then they can be applied sequentially:

$${}^A\xi_C = {}^A\xi_B \oplus {}^B\xi_C$$

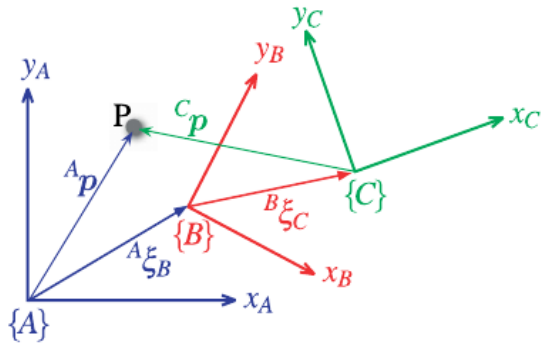
- The pose of $\{C\}$ relative to $\{A\}$ can be obtained by compounding the relative poses from $\{A\}$ to $\{B\}$ and $\{B\}$ to $\{C\}$
- We use the operator $\oplus$ to indicate composition of relative poses.



Composition of relative poses - 2

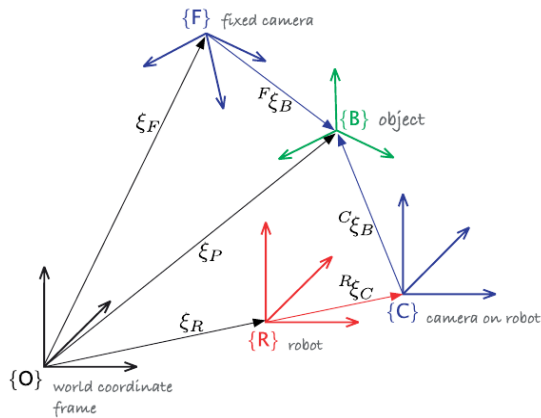
- For this case the point $\mathbf{P}$ can be described:

$${}^A\mathbf{p} = ({}^A\xi_B \oplus {}^B\xi_C) \cdot {}^C\mathbf{p}$$



3-dimensional coordinate frames

- So far we have shown 2-dimensional coordinate frames
- For other problems we require 3-dimensional coordinate frames to describe objects in our 3-dimensional world such as the pose of the end of a tool carried by a robot arm



Directed graph

- An alternative representation of the spatial relationships is a directed graph
 - Each node represents a pose
 - Each edge represents a relative pose
- We can compose relative poses using the $\oplus$ operator:

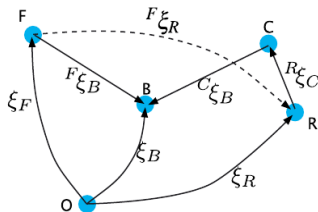
$$\xi_F \oplus {}^F\xi_B = \xi_R \oplus {}^R\xi_C \oplus {}^C\xi_B$$

$$\xi_F \oplus {}^F\xi_R = \xi_R$$

- We can subtract ξ_F from both sides of the equation by adding the inverse of ξ_F which we denote as $\ominus\xi_F$ and this gives:

$$\ominus\xi_F \oplus \xi_F \oplus {}^F\xi_R = \ominus\xi_F \oplus \xi_R$$

$${}^F\xi_R = \ominus\xi_F \oplus \xi_R$$



Representing Pose in 2-Dimensions

Cartesian coordinate system

- To represent a 2-dimensional world, we use a Cartesian coordinate system
 - Two orthogonal axes denoted x and y and typically drawn with the x-axis horizontal and the y-axis vertical
 - The point of intersection is called the origin
- Unit-vectors parallel to the axes are denoted $\hat{x}$ and $\hat{y}$
- A point is represented by its x- and y-coordinates (x, y) or as a bound vector:

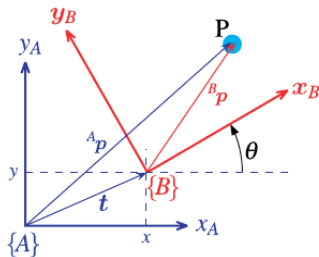
$$p = x\hat{x} + y\hat{y}$$

Representation of pose: first approach

- Let's consider a coordinate frame $\{B\}$ to describe with respect to the reference frame $\{A\}$
- The origin of $\{B\}$ has been displaced by the vector $t = (x, y)$ and then rotated counter-clockwise by an angle θ
- A concrete representation of pose is therefore the 3-vector ${}^A\xi_B \sim (x, y, \theta)$
($\sim$ denotes that the two representations are equivalent)
- Unfortunately this representation is not convenient for compounding since

$$(x_1, y_1, \theta_1) \oplus (x_2, y_2, \theta_2)$$

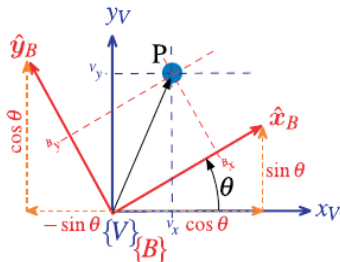
is a complex trigonometric function of both poses



Representation of pose: second approach

- Let's consider a point $\mathbf{P}$ with respect to each of the coordinate frames and to determine the relationship between ${}^A p$ and ${}^B p$
- We first consider the **rotation** problem
- We create a new frame $\{V\}$ whose axes are parallel to those of $\{A\}$ but whose origin is the same as $\{B\}$
- We can express the point P with respect to $\{V\}$ in terms of the unit-vectors that define the axes of the frame:

$$\begin{aligned} {}^V p &= {}^V x \hat{x}_V + {}^V y \hat{y}_V \\ &= \begin{pmatrix} \hat{x}_V & \hat{y}_V \end{pmatrix} \begin{pmatrix} {}^V x \\ {}^V y \end{pmatrix} \end{aligned} \quad (1)$$



Rotation

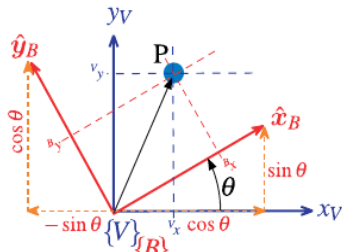
- The coordinate frame $\{B\}$ is completely described by its two orthogonal axes which we represent by two unit vectors

$$\hat{\mathbf{x}}_B = \cos \theta \hat{\mathbf{x}}_V + \sin \theta \hat{\mathbf{y}}_V$$

$$\hat{\mathbf{y}}_B = -\sin \theta \hat{\mathbf{x}}_V + \cos \theta \hat{\mathbf{y}}_V$$

- It can be factorized into matrix form as

$$\begin{pmatrix} \hat{\mathbf{x}}_B & \hat{\mathbf{y}}_B \end{pmatrix} = \begin{pmatrix} \hat{\mathbf{x}}_V & \hat{\mathbf{y}}_V \end{pmatrix} \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \quad (2)$$



- We can represent the point $\mathbf{P}$ with respect to $\{B\}$ as

$$\begin{aligned} {}^B p &= {}^B x \hat{\mathbf{x}}_B + {}^B y \hat{\mathbf{y}}_B \\ &= \begin{pmatrix} \hat{\mathbf{x}}_B & \hat{\mathbf{y}}_B \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \end{pmatrix} \end{aligned}$$

- Substituting (2) we write

$${}^V p = \begin{pmatrix} \hat{\mathbf{x}}_V & \hat{\mathbf{y}}_V \end{pmatrix} \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \end{pmatrix} \quad (3)$$

Rotation matrix

- Now by equating the coefficients of the right-hand sides of (1) and (3) we write

$$\begin{pmatrix} {}^Vx \\ {}^Vy \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} {}^Bx \\ {}^By \end{pmatrix}$$

which describes how points are transformed from frame $\{B\}$ to frame $\{V\}$ when the frame is rotated

- This type of matrix is known as a rotation matrix and denoted ${}^V\mathbf{R}_B$

$$\begin{pmatrix} {}^Vx \\ {}^Vy \end{pmatrix} = {}^V\mathbf{R}_B \begin{pmatrix} {}^Bx \\ {}^By \end{pmatrix}$$

Rotation matrix: properties

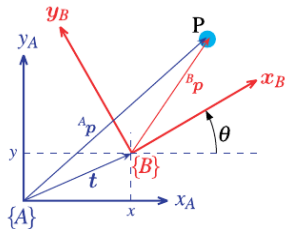
- The rotation matrix ${}^V\mathbf{R}_B$ is orthonormal
- Orthonormal matrices have the very convenient property that $\mathbf{R}^{-1} = \mathbf{R}^T$, that is, the inverse is the same as the transpose.

$$\begin{pmatrix} {}^Bx \\ {}^By \end{pmatrix} = ({}^V\mathbf{R}_B)^{-1} \begin{pmatrix} {}^Vx \\ {}^Vy \end{pmatrix} = ({}^V\mathbf{R}_B)^T \begin{pmatrix} {}^Vx \\ {}^Vy \end{pmatrix} = {}^B\mathbf{R}_V \begin{pmatrix} {}^Vx \\ {}^Vy \end{pmatrix}$$

Translation

- The second part of representing pose is to account for the translation between the origins of the frames
- Since the axes $\{V\}$ and $\{A\}$ are parallel this is simply vectorial addition

$$\begin{aligned}\begin{pmatrix} {}^A x \\ {}^A y \end{pmatrix} &= \begin{pmatrix} {}^V x \\ {}^V y \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix} \\ &= \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix} \\ &= \begin{pmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \\ 1 \end{pmatrix}\end{aligned}$$



Homogeneous transformation

- More compactly

$$\begin{pmatrix} {}^A x \\ {}^A y \\ 1 \end{pmatrix} = \begin{pmatrix} {}^A \mathbf{R}_B & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \\ 1 \end{pmatrix}$$

- $\mathbf{t} = (x, y)$ is the translation of the frame
- ${}^A \mathbf{R}_B$ is the orientation of the frame
- The coordinate vectors for point $\mathbf{P}$ are now expressed in homogeneous form

$$\begin{aligned} {}^A \tilde{\mathbf{p}} &= \begin{pmatrix} {}^A \mathbf{R}_B & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{pmatrix} {}^B \tilde{\mathbf{p}} \\ &= {}^A \mathbf{T}_B {}^B \tilde{\mathbf{p}} \end{aligned}$$

${}^A \mathbf{T}_B$ is referred to as a **homogeneous transformation**

- ${}^A\mathbf{T}_B$ represents relative pose

$$\xi(x, y, \theta) \sim \begin{pmatrix} \cos \theta & \sin \theta & x \\ -\sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{pmatrix}$$

- **First rule:** $T_1 \oplus T_2 \mapsto T_1 T_2$

$$T_1 T_2 = \begin{pmatrix} R_1 & t_1 \\ 0_{1 \times 2} & 1 \end{pmatrix} \begin{pmatrix} R_2 & t_2 \\ 0_{1 \times 2} & 1 \end{pmatrix} = \begin{pmatrix} R_1 R_2 & t_1 + R_1 t_2 \\ 0_{1 \times 2} & 1 \end{pmatrix}$$

- **Second rule:**

$$\mathbf{T}^{-1} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{t} \\ \mathbf{0}_{1 \times 2} & 1 \end{pmatrix}$$

Exercise - 1

- Spatialmath Python toolbox
- <https://github.com/petercorke/spatialmath-python>
- **SO(2): rotation in 2D**

SO(2) matrix

```
class S02(*args, **kwargs) \[source\]
```

Bases: `spatialmath.baseposematrix.BasePoseMatrix`

SO(2) matrix class

This subclass represents rotations in 2D space. Internally it is a 2x2 orthogonal matrix belonging to the group SO(2).

Exercise - 2

`__init__(arg=None, *, unit='rad', check=True)` [\[source\]](#)

Construct new SO(2) object

Parameters

- **unit** (*str, optional*) – angular units 'deg' or 'rad' [default] if applicable
- **check** (*bool*) – check for valid SO(2) elements if applicable, default to True

Returns SO(2) rotation

Return type SO2 instance

- `SO2()` is an SO2 instance representing a null rotation – the identity matrix.
- `SO2( $\theta$ )` is an SO2 instance representing a rotation by θ radians. If θ is array_like [$\theta_1, \theta_2, \dots, \theta_N$] then an SO2 instance containing a sequence of N rotations.
- `SO2( $\theta$ , unit='deg')` is an SO2 instance representing a rotation by θ degrees. If θ is array_like [$\theta_1, \theta_2, \dots, \theta_N$] then an SO2 instance containing a sequence of N rotations.

- **SE(2):** pose (rotation + translation) in 2D

SE(2) matrix

```
class SE2(*args, **kwargs) \[source\]
```

Bases: `spatialmath.pose2d.S02`

SE(2) matrix class

This subclass represents rigid-body motion (pose) in 2D space. Internally it is a 3x3 homogeneous transformation matrix belonging to the group SE(2).

Exercise - 4

```
__init__(x=None, y=None, theta=None, *, unit='rad', check=True) \[source\]
```

Construct new SE(2) object

Parameters

- **unit** (*str, optional*) – angular units 'deg' or 'rad' [default] if applicable
- **check** (*bool*) – check for valid SE(2) elements if applicable, default to True

Returns SE(2) matrix

Return type SE2 instance

- `SE2()` is an SE2 instance representing a null motion – the identity matrix
- `SE2( $\theta$ )` is an SE2 instance representing a pure rotation of θ radians
- `SE2( $\theta$ , unit='deg')` as above but θ in degrees
- `SE2(x, y)` is an SE2 instance representing a pure translation of (x , y)

Exercise - 5

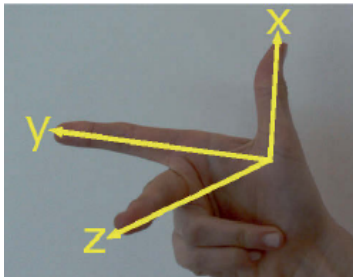
- $SO(2)$
 - Create a rotation matrix R_1 representing a $-\pi/2$ rad rotation
 - Create a rotation matrix R_2 representing a 90° rotation
 - Compute the new coordinates of $a = [1, 0]$ after a $-\pi/2$ rad rotation
 - Compute the new coordinates of $a = [1, 0]$ after a 90° rad rotation
- $SE(2)$
 - Create an homogenous matrix H_1 representing a $\pi/2$ rad rotation
 - Create an homogenous matrix H_2 representing a $[1, 2]$ translation
 - Create an homogenous matrix H_3 representing a $[1, 2]$ translation and a $\pi/2$ rad rotation
 - Compute the new coordinates of $a = [1, 0]$ after a $\pi/2$ rad rotation
 - Compute the new coordinates of $a = [1, 0]$ after a $[1, 2]$ translation
 - Compute the new coordinates of $a = [1, 0]$ after a $\pi/2$ rad rotation and a $[1, 2]$ translation

Representing Pose in 3-Dimensions

The 3-dimensional case

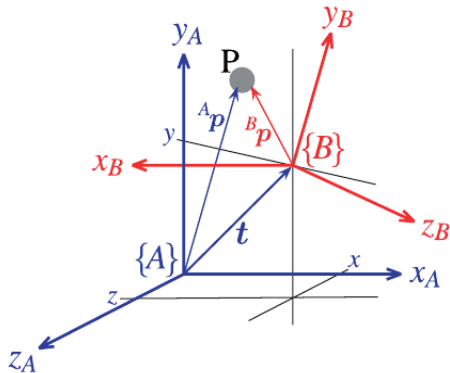
- The 3-dimensional case is an extension of the 2-dimensional
- We add an extra coordinate axis, typically denoted by z , that is orthogonal to both the x - and y -axes
- The direction of the z -axis obeys the right-hand rule and forms a right-handed coordinate frame
- A point $\mathbf{P}$ is represented by its x -, y - and z -coordinates (x, y, z) or as a bound vector

$$\mathbf{P} = x\hat{\mathbf{x}} + y\hat{\mathbf{y}} + z\hat{\mathbf{z}}$$



Representation of pose

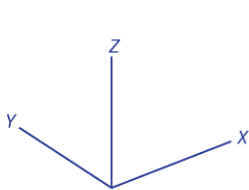
- A coordinate frame $\{B\}$ has to be described with respect to the reference frame $\{A\}$
- We can see clearly that the origin of $\{B\}$ has been displaced by the vector $\mathbf{t} = (x, y, z)$ and then rotated in some complex fashion
- Just as for the 2-dimensional case the way we represent orientation is very important



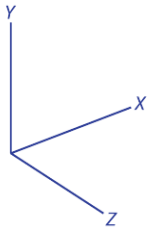
Representing Orientation in 3-Dimensions

Any two independent orthonormal coordinate frames can be related by a sequence of rotations (not more than three) about coordinate axes, where no two successive rotations may be about the same axis. Euler's rotation theorem (Kuipers 1999)

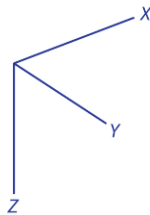
Rotation of a 3D coordinate frame



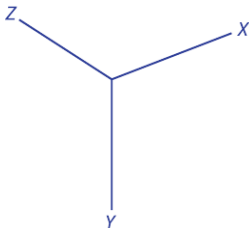
a Original



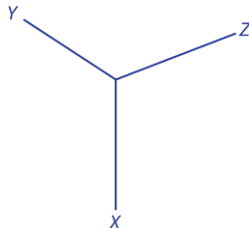
b $\frac{\pi}{2}$ about x-axis



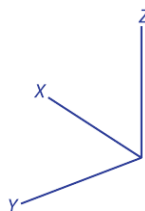
c π about x-axis



d $-\frac{\pi}{2}$ about x-axis

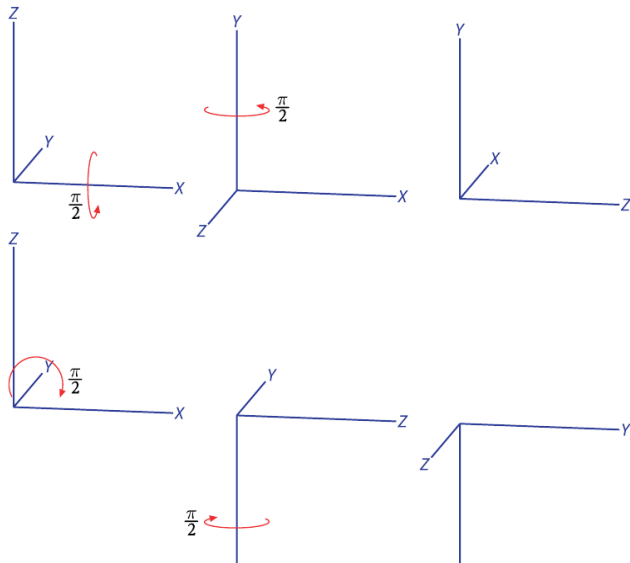


e $\frac{\pi}{2}$ about y-axis



f $\frac{\pi}{2}$ about z-axis

Example showing the non-commutativity of rotation



Example showing the non-commutivity of rotation - 2

Sequence of two rotations applied in different orders

- The final orientation depends on the order in which the rotations are applied
- This is a deep and confounding characteristic of the 3-dimensional world
- The implication for the pose algebra is that the $\oplus$ operator is not commutative

The order in which rotations are applied is very important

- There exists many ways to represent rotation
 - Orthonormal rotation matrices
 - Euler and Cardan angles
 - Rotation axis and angle
 - Unit quaternions

Orthonormal Rotation Matrix

- Just as for the 2-dimensional case we can represent the orientation of a coordinate frame by its unit vectors expressed in terms of the reference coordinate frame
- Each unit vector has three elements and they form the columns of a 3 x 3 orthonormal matrix ${}^A\mathbf{R}_B$

$$\begin{pmatrix} {}^Ax \\ {}^Ay \\ {}^Az \end{pmatrix} = {}^A\mathbf{R}_B \begin{pmatrix} {}^Bx \\ {}^By \\ {}^Bz \end{pmatrix}$$

which rotates a vector defined with respect to frame $\{B\}$ to a vector with respect to $\{A\}$

Orthonormal Rotation Matrix

- The orthonormal rotation matrices for rotation of θ about the x -, y - and z -axes are

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Exercise - 1

- **SO(3): rotation in 3D**

Constructor	rotation
SO3.Rx(theta)	about X-axis
SO3.Ry(theta)	about Y-axis
SO3.Rz(theta)	about Z-axis
SO3.RPY(rpy)	from roll-pitch-yaw angle vector
SO3.Eul(euler)	from Euler angle vector
SO3.AngVec(theta, v)	from rotation and axis
SO3.Exp(v)	from a twist vector
SO3.OA	from orientation and approach vectors

- $\text{SO}(3)$
 - Create a rotation matrix R_x representing a $\pi/2$ rad rotation around the x axis
 - Create a rotation matrix R_y representing a $\pi/2$ rad rotation around the y axis
 - Create a rotation matrix R_z representing a $\pi/2$ rad rotation around the z axis
 - Create a rotation matrix R_{xyz} representing three $\pi/2$ rad rotations around the x , y and then z axis
 - Create a rotation matrix R_{xzy} representing three $\pi/2$ rad rotations around the x , z and then y axis

Three-Angle Representations

- Euler's rotation theorem requires successive rotation about three axes such that no two successive rotations are about the same axis
- There are two classes of rotation sequence: Eulerian and Cardanian, named after Euler and Cardano respectively
- The Eulerian type involves repetition, but not successive, of rotations about one particular axis: XYX , XZX , YXY , YZY , ZXZ , or ZYZ
- The Cardanian type is characterized by rotations about all three axes: XYZ , XZY , YZX , YXZ , ZXY , or ZYX
- In common usage all these sequences are called Euler angles and there are a total of twelve to choose from

Euler angles

- It is common practice to refer to all 3-angle representations as Euler angles but this is underspecified since there are twelve different types to choose from
- The ZYZ sequence $\mathbf{R} = \mathbf{R}_z(\phi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi)$ is commonly used in aeronautics and mechanical dynamics
- Another widely used convention is the roll-pitch-yaw angle sequence angle $\mathbf{R} = \mathbf{R}_x(\theta_r)\mathbf{R}_y(\theta_p)\mathbf{R}_z(\theta_y)$ which are intuitive when describing the attitude of vehicles such as ships, aircraft and cars
- A fundamental problem with the three-angle representations just described is **singularity**
- **This occurs when the rotational axis of the middle term in the sequence becomes parallel to the rotation axis of the first or third term**

Unit Quaternions

- Quaternions were discovered by W. R. Hamilton over 150 years ago and have great utility for roboticists
- The quaternion is an extension of the complex number - a hyper-complex number - and is written as a scalar plus a vector

$$\begin{aligned}\overset{\circ}{q} &= s + \boldsymbol{v} \\ &= s + v_1i + v_2j + v_3k\end{aligned}$$

where $s \in \mathbb{R}$, $\boldsymbol{v} \in \mathbb{R}^3$ and the orthogonal complex numbers i , j and k are defined such that

$$i^2 = j^2 = k^2 = ijk = -1$$

Unit Quaternions - 2

- We denote a quaternion as

$$\overset{\circ}{q} = s \langle v_1, v_2, v_3 \rangle$$

- The quaternion or Hamilton product is

$$\overset{\circ}{q}_1 \oplus \overset{\circ}{q}_2 \mapsto s_1 s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, \langle s_1 \mathbf{v}_2 + s_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 \rangle$$

- The quaternion conjugate is

$$\ominus \overset{\circ}{q} \mapsto \overset{\circ}{q}^{-1} = s, \langle -\mathbf{v} \rangle$$

- The identity quaternion is

$$0 \mapsto 1 \langle 0, 0, 0 \rangle$$

- $\text{SO}(3)$
 - Create a rotation matrix R_{euler} representing three $\pi/2$ rad rotations using the Euler angles
 - Create a rotation matrix R_{rpy} representing three $\pi/2$ rad rotations using the Roll, Pitch, Yaw angles

Combining translation and orientation

- We return now to representing relative **pose** in three dimensions, the **position** and **orientation** change, between two coordinate frames
- The two most practical representations are:
 - the quaternion vector pair
 - 4x4 homogeneous transformation matrix
- A homogeneous transformation matrix describes rotation and translation:

$$\begin{pmatrix} {}^A x \\ {}^A y \\ {}^A z \\ 1 \end{pmatrix} = \begin{pmatrix} {}^A \mathbf{R}_B & \mathbf{t} \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \\ {}^B z \\ 1 \end{pmatrix}$$

- $\mathbf{t}$: Cartesian translation vector between the origin of the coordinates frames
- 3x3 orthonormal submatrix $\mathbf{R}$: change in orientation

Homogeneous transformation matrix

- With the vectors expressed in homogeneous form, we write:

$$\begin{aligned} {}^A\tilde{\mathbf{p}} &= \begin{pmatrix} {}^A\mathbf{R}_B & \mathbf{t} \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} {}^B\tilde{\mathbf{p}} \\ &= {}^A\mathbf{T}_B {}^B\tilde{\mathbf{p}} \end{aligned}$$

- ${}^A\mathbf{T}_B$ is a 4x4 homogeneous transformation
- Standard matrix multiplication

$$\mathbf{T}_1\mathbf{T}_2 = \begin{pmatrix} \mathbf{R}_1 & \mathbf{t}_1 \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_2 & \mathbf{t}_2 \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_1\mathbf{R}_2 & \mathbf{t}_1 + \mathbf{R}_1\mathbf{t}_2 \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix}$$

•

$$\mathbf{T}^{-1} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{R}^T & -\mathbf{R}^T\mathbf{t} \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix}$$

- SE(3)
 - Create an homogenous matrix H_{tx} representing a 1 m translation along the x axis
 - Create an homogenous matrix H_{ty} representing a 2 m translation along the y axis
 - Create an homogenous matrix H_{tz} representing a 3 m translation along the z axis
 - Create an homogenous matrix H_t representing three 1, 2 and 3 m translations along the x , y and z axis
 - Compute the new coordinates of $a = [1, 0, 0]$ after three 1, 2 and 3 m translations along the x , y and z axis
 - Create an homogenous matrix H_r representing a $\pi/2$ rad rotation around the x axis
 - Compute the new coordinates of $a = [1, 0, 0]$ after three 1, 2 and 3 m translations along the x , y and z axis and one $\pi/2$ rad rotation around the x axis